IncidentDesk — PRD & Technical Requirements

Oct 19, 2026 · @Nikhil Tiwari

Overview

IncidentDesk is a short demo of multi-agent orchestration on an enterprise IT problem: when a production incident fires, a team of specialist AI agents investigates in parallel and an Incident Commander agent turns their evidence into a root-cause report and a stakeholder update in under 2 minutes.

Problem. In a real enterprise outage, the first 30 minutes go to gathering evidence: someone reads logs, someone checks dashboards, someone asks "what changed?". That work is parallel by nature but done by people in sequence, over chat.

Why this showcases orchestration. The demo makes four orchestration patterns visible in one run: fan-out to parallel specialists, a shared evidence board they all write to, a coordinator that waits for everyone before deciding, and a human approval gate before anything is "actioned".

Scope. A demo, not a product. The enterprise is a fictional retailer, ShopCo, with three services (checkout, payments, inventory). All logs, metrics and change records are synthetic and generated by a script, so no real company data is ever sent to a model.

Layer	Choice	Role
Agent framework	Google Agent Development Kit (ADK), Python	Agents, tools, `ParallelAgent` and `SequentialAgent` workflows, shared session state
Model provider	Groq free tier, via ADK's `LiteLlm` wrapper	Every LLM call
Orchestration	Scion (Workstation mode)	Each specialist as its own containerised agent, shared directory as the evidence board, dashboard to watch the team
Data	Synthetic ShopCo telemetry (JSONL logs, CSV metrics, JSON change log, Markdown runbooks)	Everything the agents investigate

Layer	Choice	Role
Version control	Git + GitHub	Code, scenarios and saved incident reports

Goals, non-goals and demo success criteria

The demo succeeds if a viewer watches three agents investigate at once, sees the commander name the right root cause with evidence, and it costs ₹0.

Goals

- Show parallel specialist agents and a coordinating agent working on one incident.
- Produce a grounded incident report: every claim cites a log line, metric or change record.
- Run the same team two ways: in-process with ADK, and as separate Scion agents.
- Keep a human approval step before any remediation is marked as done.
- Fit comfortably inside Groq's free tier.

Non-goals

- Connecting to real monitoring tools (Datadog, PagerDuty, ServiceNow); real integrations are a later step.
- Executing remediations; actions are recommended and approved, never run.
- A production UI; the interface is the terminal, the Scion dashboard and a Markdown report.
- More than three scenarios.

Demo success criteria

Criterion	Target
Correct root cause, 3 scripted scenarios	3 of 3
Time from alert to report	< 2 minutes
Claims in report backed by cited evidence	100%
Groq calls per incident	⩽ 10
Monthly cost	₹0
Live demo length	⩽ 5 minutes

The demo

One command fires a synthetic outage at ShopCo; the audience watches the agent team investigate on the Scion dashboard and gets a report they can check against the planted fault.

The agent team

Agent	Looks at	Answers
Log Analyst	Application logs (JSONL)	Which errors spiked, where, and since when?
Metrics Analyst	Latency, error rate, CPU, DB connections (CSV)	Which signals went abnormal first?
Change Auditor	Deploys, config changes, certificate expiries (JSON)	What changed shortly before the incident?
Incident Commander	The evidence board + runbooks	Root cause, confidence, timeline, recommended actions
Comms Writer	The Commander's report	A plain-English update for business stakeholders

Five-minute storyline

1. **Alert (0:00).** `make incident SCENARIO=bad-deploy` generates fresh ShopCo telemetry with a planted fault and fires an alert: "checkout error rate 14%".
2. **Fan-out (0:15).** Three specialist agents start at once; the Scion dashboard shows all three thinking and executing.
3. **Evidence board (0:45).** Each specialist writes its findings to the shared board; the audience sees the files appear.
4. **Synthesis (1:15).** The Commander, which waited for all three, correlates the evidence and drafts the report.
5. **Human gate (1:45).** The presenter approves the recommended rollback with `scion message`; the Comms Writer then drafts the stakeholder update.
6. **Reveal (3:00).** The report is compared with the scenario's ground-truth file, live.

Scenarios (each planted by the generator, with a ground-truth answer)

Scenario	Planted fault	What a correct report says
`bad-deploy`	Payments v2.4.1 deploy shrinks the DB connection pool	Deploy at 14:02 caused pool exhaustion; roll back to v2.4.0

Scenario	Planted fault	What a correct report says
`cert-expiry`	Inventory's TLS certificate expires	Expired cert broke checkout→inventory calls; renew and reload
`noisy-neighbour`	A batch job saturates the shared database CPU	Batch job at 13:55 starved checkout queries; reschedule the job

Functional requirements

Each requirement is testable; all are P0 for the demo unless marked.

ID	Requirement	Priority
FR-1	A generator script creates 60 minutes of ShopCo telemetry for a named scenario: logs, metrics, change log, plus a hidden `ground_truth.json`	P0
FR-2	An alert (service, symptom, start time) starts the investigation	P0
FR-3	Log Analyst, Metrics Analyst and Change Auditor run in parallel, each with read-only tools over its own data source	P0
FR-4	Each specialist writes one finding file to the evidence board: summary, key evidence (with source and timestamp), confidence 0–1	P0
FR-5	The Incident Commander starts only when all three findings exist, or after a 90 s timeout with what is available	P0
FR-6	The Commander produces the incident report: summary, timeline, root cause, confidence, evidence table, recommended actions from runbooks	P0
FR-7	Every claim in the report cites a finding; claims without evidence are labelled "hypothesis"	P0
FR-8	Recommended actions require human approval before the report marks them "approved"; nothing is ever executed	P0

ID	Requirement	Priority
FR-9	The Comms Writer produces a stakeholder update of ⩽ 120 words with no technical jargon	P1
FR-10	A scorer compares the report's root cause with `ground_truth.json` and prints pass/fail	P1
FR-11	The same agent team runs in two modes: single-process ADK (`adk web`) and Scion multi-agent	P0
FR-12	Specialists report status to Scion (`thinking`, `executing`, `completed`) so the dashboard shows progress	P1

Finding file format (one per specialist)

```
{
  "agent": "change_auditor",
  "summary": "payments v2.4.1 deployed 14:02, 3 min before errors began",
  "evidence": [
    {"source": "changes.json", "ts": "14:02:10", "detail": "deploy payments
v2.4.0 -> v2.4.1"}
  ],
  "confidence": 0.8
}
```

Incident report sections: Summary · Impact · Timeline · Root cause (with confidence) · Evidence table · Recommended actions (approval status) · Stakeholder update.

Free-tier calibration

One incident uses at most 10 Groq calls and about 12,000 tokens, so even dozens of demo rehearsals a day stay free; the one thing to watch is the tokens-per-minute cap during the parallel burst.

Groq's free tier needs no credit card and is limited by rate, set per model and applied per organisation. Published figures vary (roughly 30 requests/minute and 1,000+ requests/day, with tokens-per-minute caps as low as 6,000 on some models). **Read the real limits for your chosen models at console.groq.com/settings/limits** and put them in `config.py`.

Step	Groq calls	Input tokens (cap)	Model tier
3 specialists, in parallel	⩽ 6 (⩽ 2 each: tool call + answer)	⩽ 2,000 each	Small, fast model
Incident Commander	⩽ 2	⩽ 4,000	Larger model
Comms Writer	1	⩽ 1,000	Small, fast model
Per incident	**⩽ 10**	**≈ 12,000**	

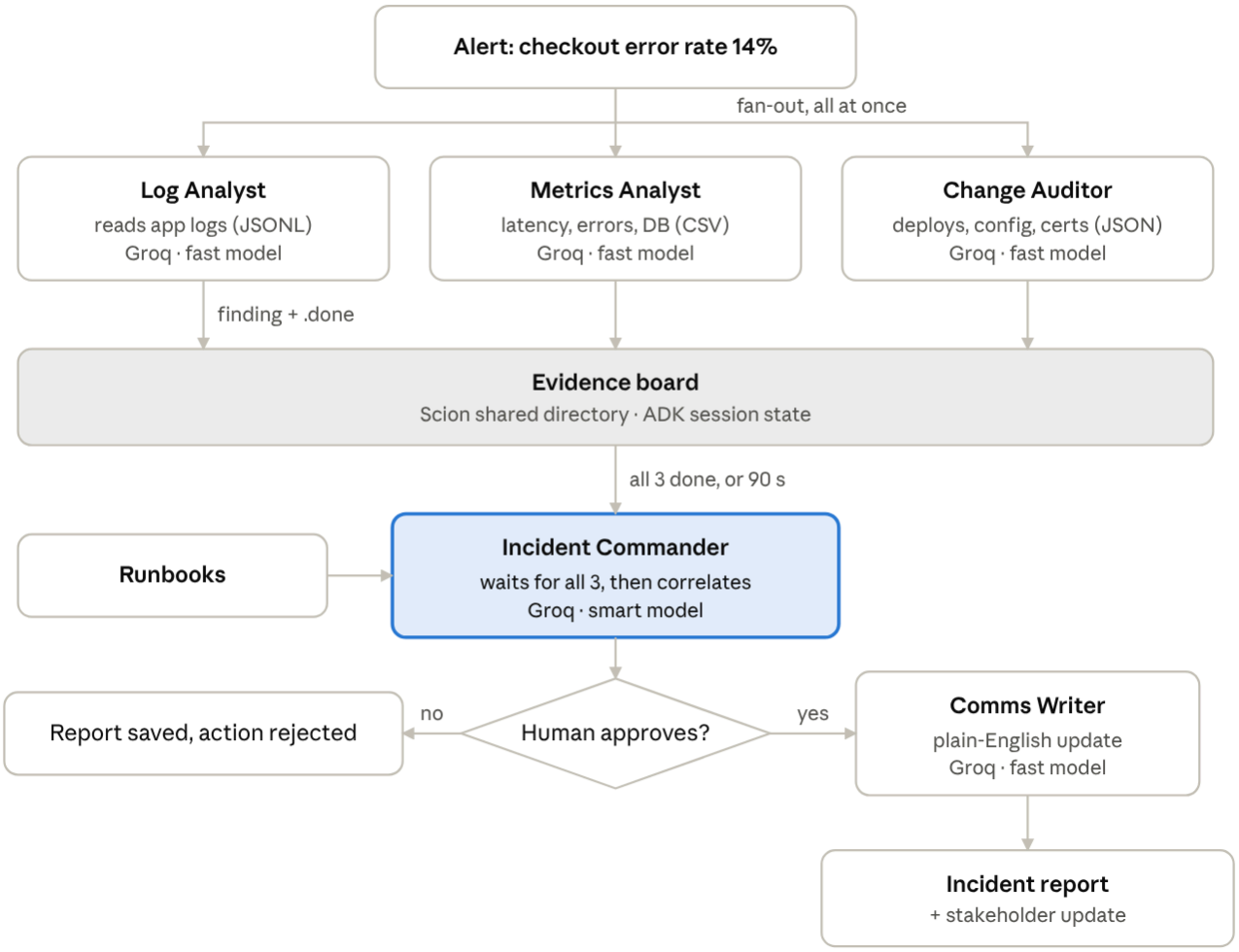
Design rules

1. Tools do the heavy lifting in plain Python: filter by time window, aggregate, and return compact summaries capped at 1,500 tokens. The LLM never reads raw telemetry.
2. The LLM only judges and writes: what the evidence means, and how to explain it.
3. If the chosen models' TPM cap is below the 6,000-token parallel burst, specialists start 3 s apart instead of together.
4. On HTTP 429: retry 3 times with backoff 5 s / 15 s / 45 s, honouring `retry-after`, then fall back to a second Groq model.
5. **Synthetic data only.** Groq's free tier offers no data-privacy guarantee, and the demo needs none: every input is generated.
6. Model IDs live in config, so a retired Groq model is a one-line change.

Architecture

The team is a fan-out / fan-in workflow: three specialists run at once, write to a shared evidence board, and the commander decides only after all have reported.

Three specialists work in parallel; the commander decides after all report



IncidentDesk agent team · fan-out, evidence board, commander, approval gate

The same team runs in two modes. In **ADK mode**, a `ParallelAgent` runs the specialists in one process and the evidence board is ADK session state. In **Scion mode**, each box is its own container and the board is a Scion shared directory, which is the version to demo.

Repository layout

```
incidentdesk/
├── incidentdesk/
|   ├── __main__.py      # --role, --input; runs one role or the full team
|   ├── agent.py         # specialists, commander, comms, root_agent
|   ├── tools.py         # error_summary, metric_anomalies, list_changes,
read_runbook
|   ├── schemas.py       # Finding, Report (Pydantic)
|   ├── board.py         # evidence board: state or shared-dir files + .done
markers
|   └── config.py        # models, limits, retry policy
├── generate.py          # synthetic ShopCo telemetry + ground truth
├── runbooks/            # rollback.md, cert-renewal.md, batch-jobs.md
├── scenarios/           # bad-deploy, cert-expiry, noisy-neighbour
├── templates/           # one Scion template per role
├── tests/               # pytest + eval scorer
├── demo.sh
├── Dockerfile           # from scion-base
└── requirements.txt     # pinned
```

Technical requirements

The build is Python 3.11+, Google ADK with every model behind `LiteLlm` on Groq, deterministic Python tools over synthetic files, and a Scion container built from the official ADK example.

Runtime and models

ID	Requirement
TR-1	Python 3.11+; `google-adk`, `litellm`, `pydantic`, `python-dotenv`, pinned to exact versions
TR-2	All agents use `LiteLlm(model=...)` from `google.adk.models.lite_llm`; no Gemini-only ADK tools
TR-3	Two model slots in `config.py`: `FAST_MODEL` (specialists, comms) and `SMART_MODEL` (commander), e.g. `groq/llama-3.1-8b-instant` and `groq/llama-3.3-70b-versatile`; confirm both in the Groq console first
TR-4	Temperature 0.1 for specialists and commander, 0.5 for comms; `max_tokens` 800 / 1,500 / 400

ID	Requirement
TR-5	Secrets only via environment: `GROQ_API_KEY`; `.env` gitignored

Agent design (ADK)

ID	Requirement
TR-6	`root_agent = SequentialAgent([investigators, commander, comms])`, where `investigators = ParallelAgent([log_analyst, metrics_analyst, change_auditor])`
TR-7	Specialists write findings via `output_key` (`log_findings`, `metric_findings`, `change_findings`); the commander reads them through `{placeholders}` in its instruction
TR-8	Findings and the report are validated against Pydantic schemas; an invalid output gets one repair retry
TR-9	The approval gate is a tool, `request_approval(action)`, that pauses for a human yes/no: terminal input in ADK mode, `scion message` in Scion mode
TR-10	`adk web` must run the whole team locally for debugging and for the in-process demo mode

Tools (plain Python, read-only)

Tool	Returns
`error_summary(service, start, end)`	Error counts per minute and the top 5 error messages, with first-seen times
`metric_anomalies(service, start, end)`	Metrics that left their baseline band, with when and by how much
`list_changes(start, end)`	Deploys, config changes, scheduled jobs, certificate events in the window

Tool	Returns
`read_runbook(topic)`	The matching runbook section (rollback, cert renewal, job rescheduling)

Synthetic data and testing

ID	Requirement
TR-11	`generate.py --scenario <name> --seed <n>` is deterministic for a given seed and writes `data/<run-id>/`
TR-12	Normal traffic includes background noise (routine warnings, unrelated deploys) so the agents must discriminate, not just find the only error
TR-13	Unit tests (pytest) for the generator and every tool; no Groq calls in unit tests
TR-14	`make eval` runs all 3 scenarios × 3 seeds against ground truth and prints a score table; run manually, as it uses Groq quota

Scion orchestration

In Scion mode each role runs as its own containerised agent, and a Scion shared directory acts as the evidence board, so the audience sees a real team of agents rather than one process.

ID	Requirement
SR-1	Scion installed in Workstation mode; `scion init` in the repo root; Git ⩾ 2.48 and Docker on the host
SR-2	One container image built on the pattern of Scion's `examples/adk_scion_agent/` (base `scion-base`), with `litellm` added; entry point `python -m incidentdesk --role <role>`
SR-3	One Scion template per role (`log-analyst`, `metrics-analyst`, `change-auditor`, `commander`, `comms`), all on the **generic** harness with `task_flag: "--input"`, sharing the one image

ID	Requirement
SR-4	A shared directory `evidence`, created with `scion shared-dir create evidence`, mounted in every agent at `/scion-volumes/evidence`
SR-5	Coordination by files: each specialist writes `<role>.json` then touches `<role>.done`; the commander waits for all three `.done` markers or 90 s
SR-6	The commander writes `report.md` to the board and pauses at the approval gate; the presenter answers with `scion message commander "approve"`
SR-7	`GROQ_API_KEY` stored as a Hub project secret, never in the image or templates
SR-8	Specialists call `sciontool status` so the dashboard shows each agent's state live

Demo run script (`demo.sh`)

```
SCENARIO=${1:-bad-deploy}
python generate.py --scenario $SCENARIO --seed 7 --out "$EVIDENCE_DIR/data"
# EVIDENCE_DIR = host path from `scion shared-dir info evidence`
ALERT="checkout error rate 14% since 14:05 — investigate"

scion start log-analyst     --template log-analyst     "$ALERT" &
scion start metrics-analyst --template metrics-analyst "$ALERT" &
scion start change-auditor  --template change-auditor  "$ALERT" &
scion start commander       --template commander       "$ALERT"
wait
# then: watch the dashboard, approve with `scion message commander
"approve"`,
# and start `comms` once report.md exists
```

The exact path the host uses for the shared directory is checked in M3; the script writes telemetry wherever `scion shared-dir info evidence` says the directory lives.

Risks and mitigations

The main risks are a wrong root cause on stage and free-tier limits during the parallel burst; both are rehearsed away before the demo.

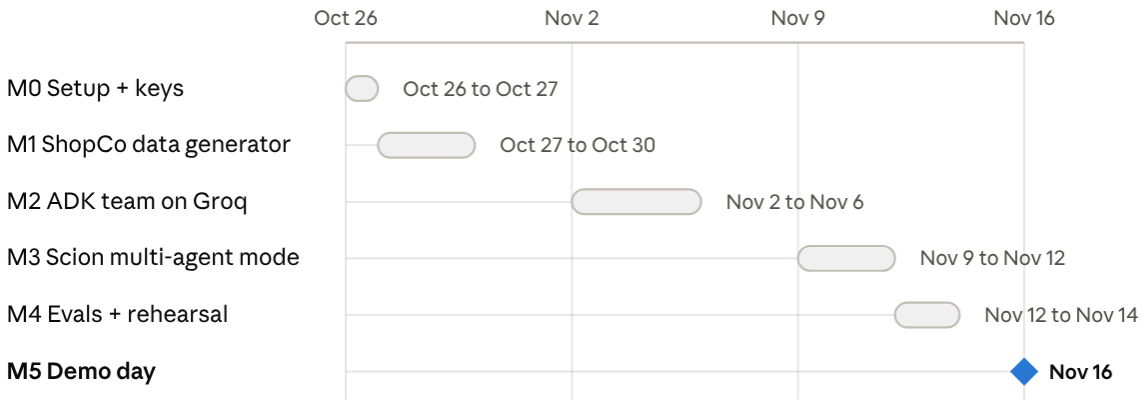
Risk	Impact	Mitigation
Commander names the wrong root cause live	Demo fails its key moment	Fixed seeds rehearsed via `make eval`; background noise tuned so scenarios are solvable but not trivial
Parallel burst exceeds tokens-per-minute cap	429 errors mid-demo	Tool outputs capped at 1,500 tokens; staggered starts; retries and fallback model
Small model produces malformed tool calls	Specialist stalls	Use the larger model for any specialist that fails evals; schema repair retry
Commander waits forever for a stuck specialist	Demo hangs	90 s timeout; report notes which evidence is missing
Groq retires a model or changes free limits	Build breaks	Model IDs in config; LiteLLM lets the provider switch without code changes
Scion is new and changes weekly	Setup breaks before demo	Pin the Scion version; the ADK-only mode is always a working fallback
Scion's ADK example builds `google-adk` from git HEAD	Unstable image	IncidentDesk does not use the example's `EnvironmentToolset`, so it pins a released `google-adk`
Audience assumes it touched real systems	Misleading demo	Say "synthetic ShopCo data" on the first slide and in every report header

Milestones and open questions

The demo is ready in about three weeks part-time, with a working ADK-only version by the end of week 2 as the safety net.

Demo-ready in three weeks

IncidentDesk plan by week, 2026 (proposed dates)



IncidentDesk milestones · 3 weeks

Milestone	Exit check
M0 Setup + keys	Groq key works from ADK via `LiteLlm`; Git ≥ 2.48 and Docker running; Groq limits recorded in `config.py`
M1 ShopCo data generator	All 3 scenarios generate deterministically with noise; tool unit tests pass
M2 ADK team on Groq	`adk web` solves all 3 scenarios in ADK mode, ≤ 10 calls each
M3 Scion multi-agent mode	`demo.sh` runs 5 agents, evidence board fills, approval works via `scion message`
M4 Evals + rehearsal	`make eval` scores 9 of 9 (3 scenarios × 3 seeds); two timed dry runs under 5 minutes
M5 Demo day	Live run, root cause revealed against ground truth

Open questions

- [ ] Exact Groq free limits (RPM, RPD, TPM) for the two chosen models.
- [ ] Is `llama-3.1-8b-instant` reliable enough at tool calling for the specialists, or do they need the larger model?
- [ ] Host path of the `evidence` shared directory in Workstation mode, for `demo.sh`.
- [ ] Who is the demo audience (recruiters, a class, a team), which sets how technical the narration is?

Sources

- Scion overview
- Scion: Agent Development Kit
- Scion: About Workspaces (shared directories, coordination files)
- Scion ADK example README
- ADK LiteLLM provider patterns (adk-python PR 4434)
- Groq free-tier overview (pricepertoken.com) and Groq limits summary (costbench.com)